Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I believe Sequences, Nested Data, and the Crypto Corgi project can help me complete this project, and I expect to use all of them. I also want to improve my For Loop skills while completing this project.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I expect the easiest part will be designing the project, and the hardest might be correctly expressing my ideas and implementing them into AI.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

Tasks that will require the most time and effort might be implementing the encryption key to be generated along with the input message. The first implemented feature

should be the encryption of the input message.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Distractions could include work from other classes, since the end of the semester brings a lot of work from all classes with final projects and exams. I will minimize these distractions by having a set schedule to work on different assignments.

1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

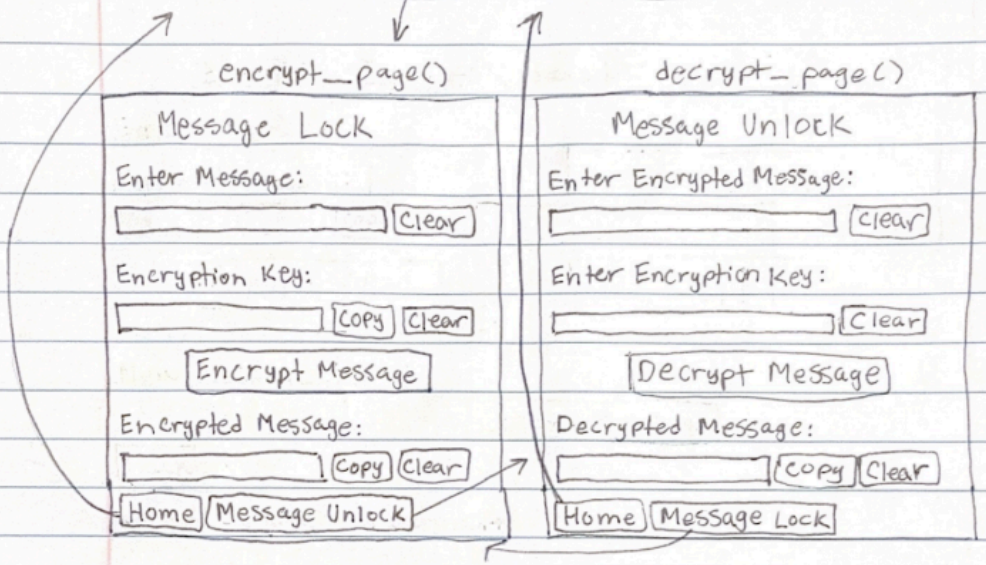
Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.

home_page()

MESSAGE ENCRYPTION

Encrypt and Decrypt securely!

Get Started



@dataclass

Class State:

All page functions will have a state parameter

message: str (original message)

encryption_key: str (randomly generated key)

encrypted_message: str (encrypted message)

decrypted_message: str (decrypted message)

error_message: str (errors for empty input/incorrect key)

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	To provide users with a simple way to encrypt/decrypt their messages with randomly generated encryption keys.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	State Attributes: current_message (str): the current original message input current_key (str): the current encryption key used current_encrypted (str): the current encrypted message Current_decrypted (str): the current decrypted message History (list[EncryptionRecord]): keeps track of past encryptions
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	EncryptionRecord Attributes: original_message (str): the original message input encryption_key (str): the encryption key that is generated along with the encrypted message encrypted_message (str): the encrypted message date_created (str): the date the encrypted message was created
Routes (minimum 7)	home_page: welcome page encrypt_page: encryption interface

The names of your route functions	decrypt_page: decryption interface history_page: previous encryption records help_page: help resources encrypt_message: encrypt's user's message and saves a record decrypt_message: decrypts the message using the given encryption key
Pages (minimum 5) A brief description of the pages in your site	Home page: introduces the purpose and explains encryption/decryption, navigation buttons for encryption, decryption, and history Encrypt page: Textbox for original message, generate key button, displays generated key, encrypt button, displays encrypted message Decrypt page: textbox for encrypted message and encryption key, decrypt button, displays original message History page: displays previous encrypted messages, displays each encrypted version and supplied key, allows review of previous encryption activities Help page: explains how to generate a key, instructions for encrypting and decrypting, reminder to save generated keys
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	Enter Message: original message to encrypt Encrypted Message: encrypted message to recover Encryption Key: generated key during encryption
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully	Check for an empty message: calls error_message and produces a string: "Please enter a message" Check if a key exists: calls error_message and produces a string: "Please generate an encryption

change the logic of the site	key first” Check if the encryption key in correct: if correct, decrypts the message like normal, if incorrect, calls error_message and produces a string: “Incorrect encryption key” Check if history has records: if not empty, displays the encryption history like normal, if empty, produces a string: “No previous messages”
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	For a record in the history section, print “Original:”, then the original message, “Encrypted:”, then the encrypted message, “Key:”, then the encryption key, and “Date:”, then the date created

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

here is a mock-up of my planned website. Can you give parameters for home_page(), encrypt_page, and decrypt_page(), as well as what fields my 'State' will have?

Absolutely! I can help you plan the Drafter structure and keep it simple enough for a CS1 project.

One issue: your uploaded mock-up appears to be a scanned PDF, and I wasn't able to read the actual page designs. If you upload screenshots or images of the mock-up, I can tailor these recommendations to your exact layout.

In the meantime, here's the structure I'd recommend.

1. Page functions and parameters

In Drafter, each page function should receive a `state` parameter. The state contains the information your application needs to remember as users navigate between pages.

Function	Parameters	Purpose
`home_page()`	`state`	Displays the welcome page and navigation to encryption and decryption.
`encrypt_page()`	`state`	Displays the message input, key generator, and encryption button.
`decrypt_page()`	`state`	Displays the encrypted message input, key input, and decryption button.

Your function signatures would look like:

```
def home_page(state: State) -> Page:
    ...

def encrypt_page(state: State) -> Page:
    ...

def decrypt_page(state: State) -> Page:
    ...
```

2. Recommended State fields

I recommend starting with these five fields:

State field	Data type	Purpose
`message`	`str`	Stores the original message entered by the user.
`encryption_key`	`str`	Stores the randomly generated encryption key.
`encrypted_message`	`str`	Stores the encrypted version of the message.
`decrypted_message`	`str`	Stores the decrypted result.
`error_message`	`str`	Displays errors, such as an incorrect key or missing input.

For example:

```
@dataclass

class State:

  message: str

  encryption_key: str

  encrypted_message: str

  decrypted_message: str

  error_message: str
```

You would initialize these fields with empty strings when creating your initial state.

3. How the pages interact

Here's the basic navigation and information flow:

Home Page

Choose an action

Encrypt Page

Enter message

Generate key

Encrypt message

Decrypt Page

Enter encrypted message

Enter key

Decrypt message

4. A few recommendations

To keep the project manageable:

- Don't create separate State fields for every button or text box.
- Don't store information that can easily be calculated again.
- Keep encryption and decryption logic in separate helper functions rather than putting all the logic inside your page functions.
- Use a cryptographically secure random key generator and a vetted encryption library if you intend to protect real messages. A homemade cipher is suitable only as an educational demonstration.

One important design decision: Your encryption key must be available to the user after encryption so they can use that same key when decrypting the message.

5. What I'd like to see

Please upload your mock-up as an image or screenshot so I can check the actual layout.

Then I can give you a more precise breakdown of:

- The exact parameters and State fields needed for your design.
- Which Drafter components each page needs.
- Which buttons need callback functions.
- How information should move between the three pages.

can you give implementations for the following: purpose/functionality of the website, 4 dataclass attributes with one as a list/dict of instances in the second dataclass, a second dataclass with attributes, 7 route functions, description of the pages (minimum 5), description of the user input textboxes, if statements (minimum 3), and at least 1 loop that iterates over the list/dict of dataclass instances in the State attribute

Message Encryption Tool – Project Implementation Plan

Below is a proposed implementation plan for your Drafter website. I've organized it around each requirement in your project assignment.

The goal is to build a beginner-friendly encryption website that allows users to generate random keys, encrypt messages, decrypt messages, and keep a history of their previous encryption activities.

I'll keep the design realistic for a three-week CS1 project using Python and Drafter.

1. Purpose and functionality of the website

Project purpose:

The purpose of this website is to provide users with a simple way to encrypt and decrypt text messages using randomly generated encryption keys. Users can enter a message, generate a unique key, encrypt the message, and save the result. They can later enter an encrypted message and its corresponding key to recover the original message.

Main functionality:

- Generate random encryption keys.
- Encrypt messages using a generated key.
- Decrypt messages using the correct key.
- Copy encrypted messages and keys.
- View a history of previously encrypted messages.

2. Dataclasses and State attributes

I recommend using two dataclasses. The first represents the overall website state, while the second represents an individual encryption record.

Dataclass 1: EncryptionRecord

Each record represents one encryption activity.

```
@dataclass

class EncryptionRecord:
```

```
    original_message: str
    encrypted_message: str
    encryption_key: str
    date_created: str
```

Dataclass 2: State

Your State stores the current information being used by the website.

```
@dataclass
class State:
    current_message: str
    current_key: str
    current_encrypted_message: str
    current_decrypted_message: str
    history: list[EncryptionRecord]
```

Meets the dataclass requirement

You have:

- Four regular State attributes.
- One list attribute containing instances of `EncryptionRecord`.
- Four attributes in the second dataclass.

The `history` list allows users to keep track of multiple encryption activities.

3. Seven route functions

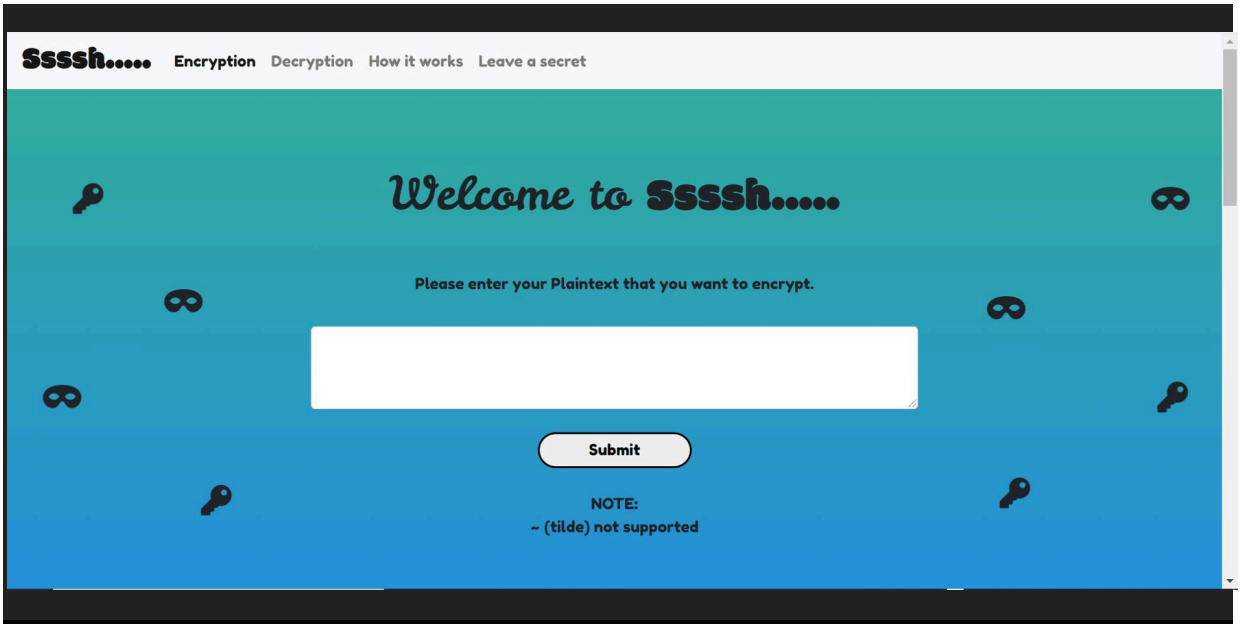
I recommend these seven routes:

Route function	Purpose
`home_page`	Displays the welcome page.
`encrypt_page`	Displays the encryption interface.
`decrypt_page`	Displays the decryption interface.
`history_page`	Displays previous encryption records.
`generate_key`	Generates a random encryption key.
`encrypt_message`	Encrypts the user's message and saves a record.
`decrypt_message`	Decrypts the message using the supplied key.

The first four routes display pages. The last three perform actions and update the State.

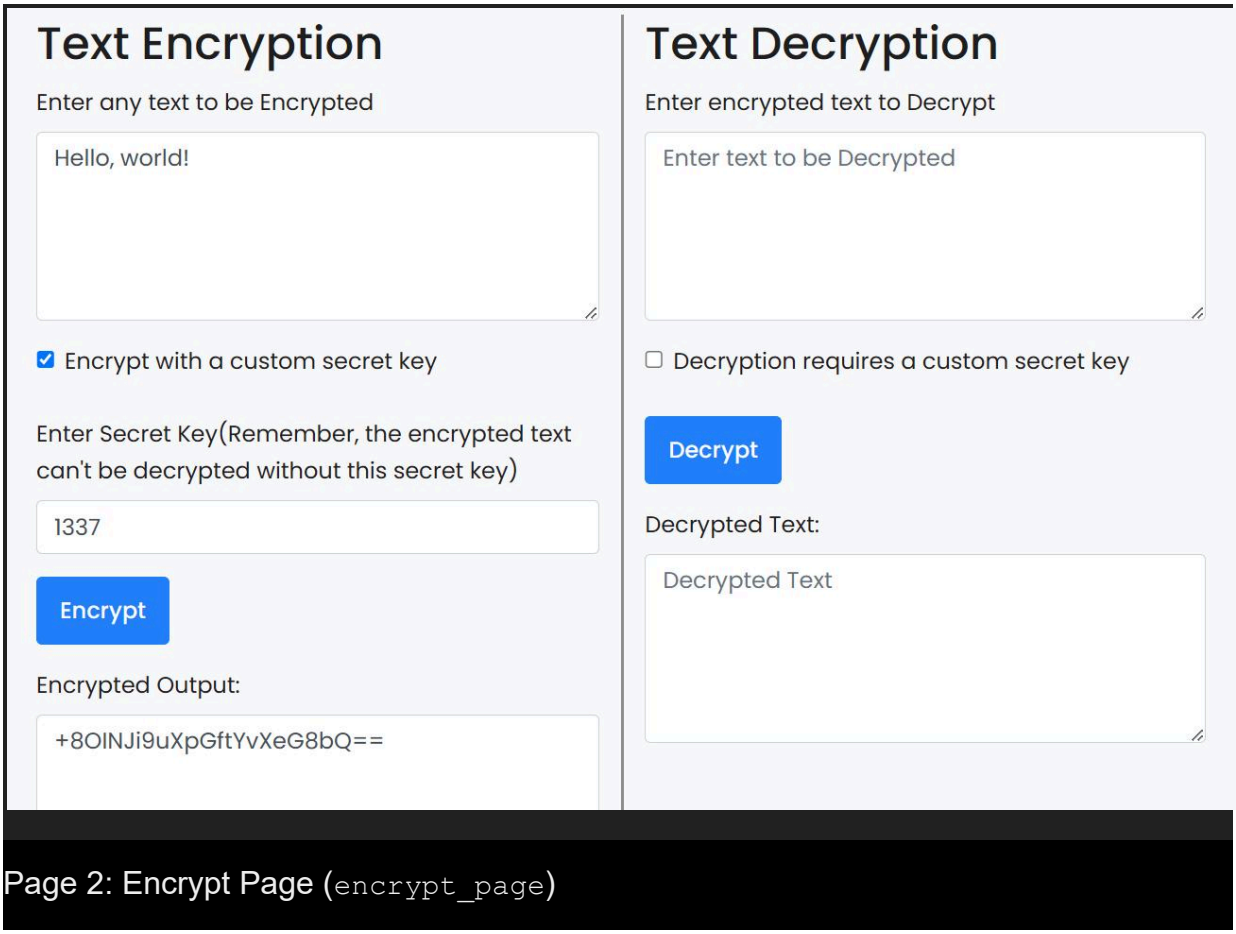
4. Description of the website pages (minimum 5)

I recommend building five pages. This gives your website enough functionality without making it too complicated.



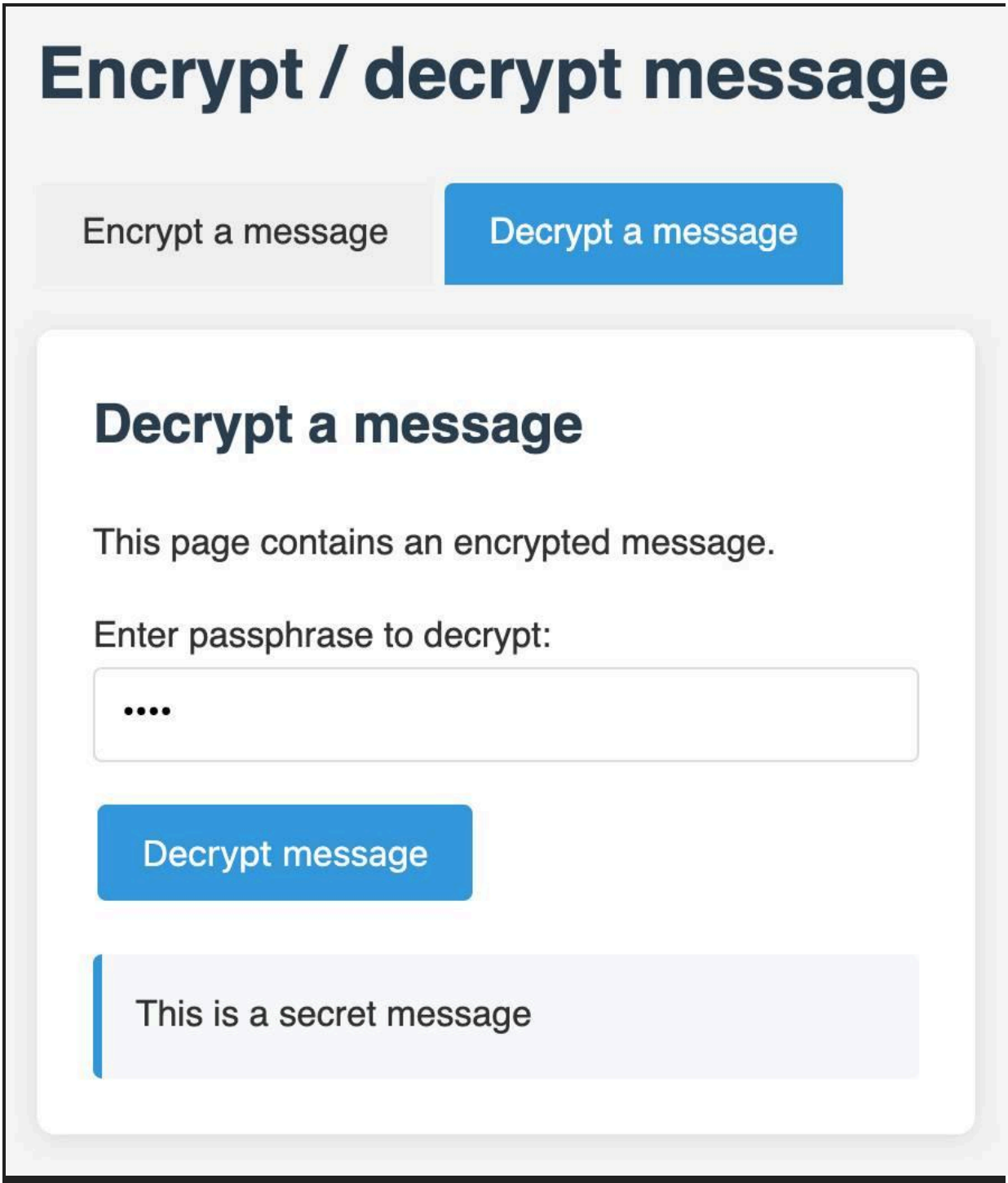
Page 1: Home Page (`home_page`)

- Introduces the purpose of the website.
- Explains what encryption and decryption do.
- Has navigation buttons for Encrypt, Decrypt, and History.



Page 2: Encrypt Page (`encrypt_page`)

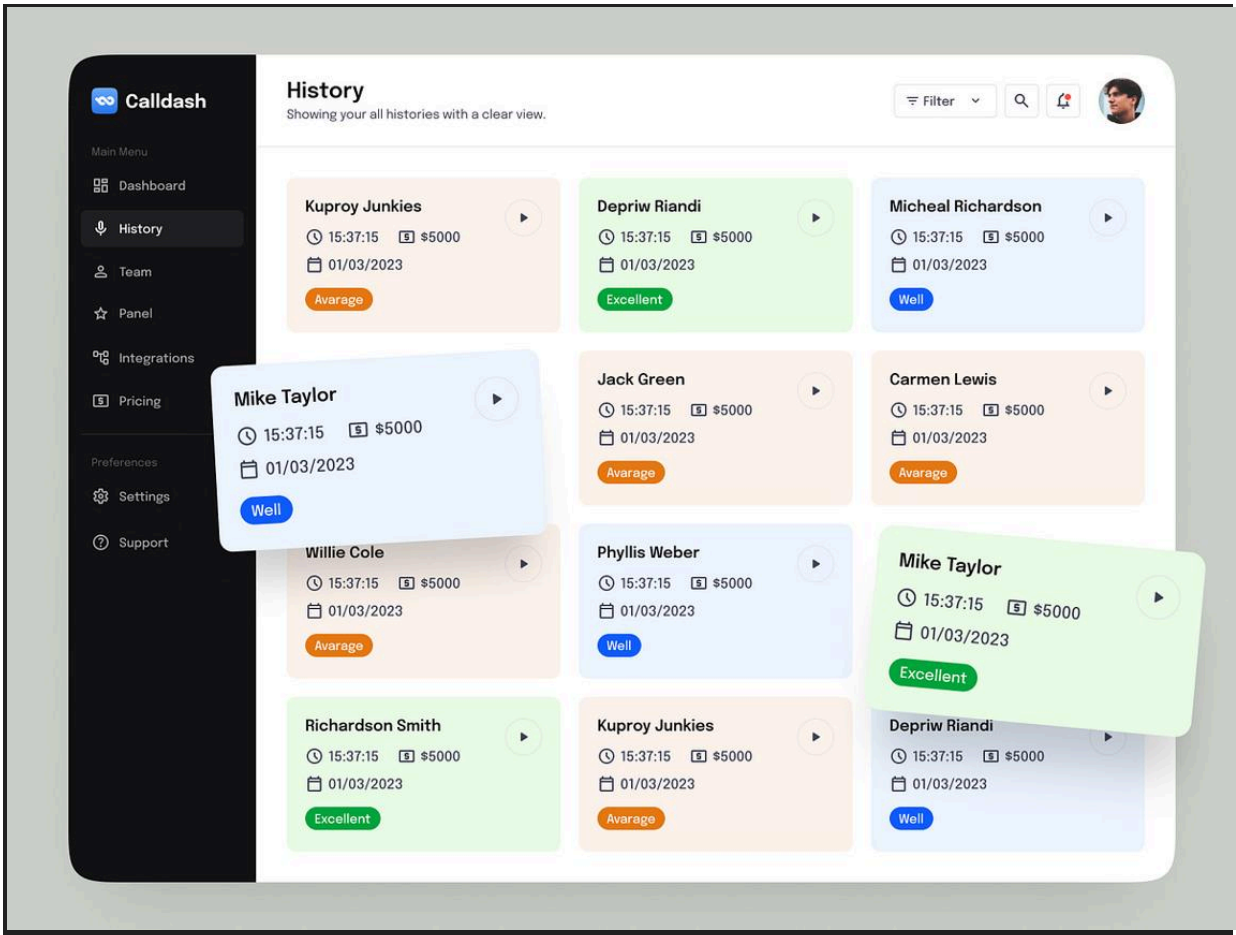
- Contains a textbox for the original message.
- Has a Generate Key button.
- Displays the generated encryption key.
- Has an Encrypt button.
- Displays the encrypted message.



Page 3: Decrypt Page (`decrypt_page`)

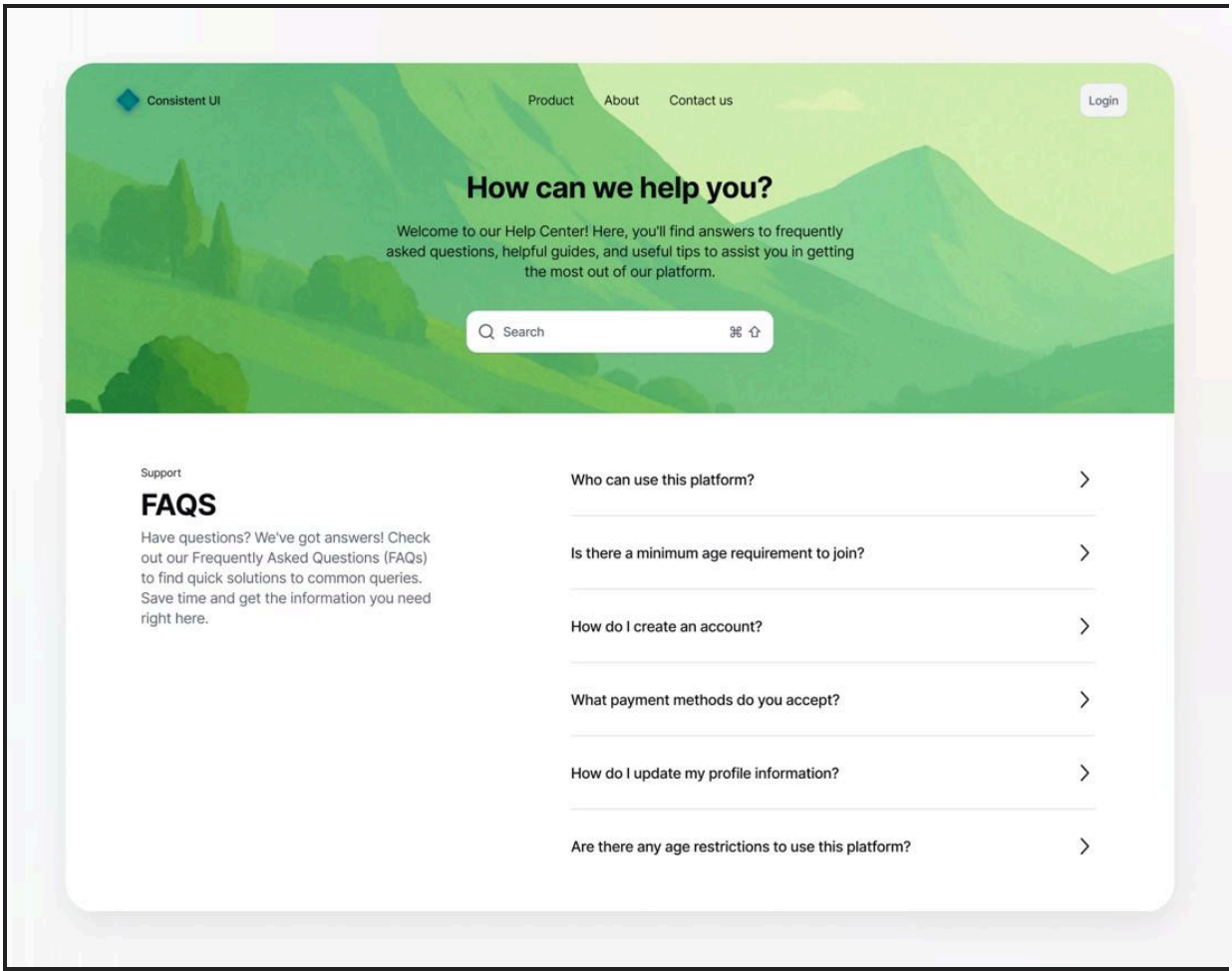
- Contains a textbox for the encrypted message.

- Contains a textbox for the encryption key.
- Has a Decrypt button.
- Displays the recovered original message.



Page 4: History Page (`history_page`)

- Displays previously encrypted messages.
- Shows each message's encrypted version and key.
- Allows users to review previous encryption activities.



Page 5: Help Page (`help_page`)

- Explains how to generate a key.
- Provides instructions for encrypting messages.
- Explains how to decrypt messages.
- Reminds users to save their keys.

5. Description of user input textboxes

Your website should have four main textboxes.

Textbox	Page	What the user enters

Original Message	Encrypt	The regular message they want to encrypt.
Encrypted Message	Decrypt	The encrypted text they want to recover.
Encryption Key	Decrypt	The key originally generated during encryption.
Custom Key (optional)	Encrypt	An optional key if the user wants to enter their own instead of generating one.

My recommendation: Make the custom key optional. Your main feature should still be generating random keys automatically.

You can also use buttons rather than textboxes for generating keys and copying results.

6. If statements (minimum 3)

Here are four examples of conditional logic that would be useful in your application.

If statement 1: Check for an empty message

```
if state.current_message == "":
    state.error_message = "Please enter a message."
```

This prevents users from encrypting an empty message.

If statement 2: Check whether a key exists

```
if state.current_key == "":
```

```
    state.error_message = "Please generate an encryption key first."
```

This ensures that users have a key before attempting encryption.

If statement 3: Check whether the decryption key is correct

```
if entered_key == state.current_key:
    # Decrypt the message
    state.current_result = decrypt_text(
        encrypted_message, entered_key
    )
else:
    state.error_message = "Incorrect encryption key."
```

This checks whether the user entered the correct key.

Important: For a more complete implementation, the key should be validated by attempting authenticated decryption, rather than comparing against only the current key in State. That also allows users to decrypt messages from previous sessions.

If statement 4: Check whether history contains records

```
if len(state.history) == 0:
    # Display "No previous messages."
else:
    # Display the encryption history.
```

This prevents the history page from appearing empty without explanation.

7. Loop that iterates through dataclass instances

Your `history` attribute contains a list of `EncryptionRecord` objects.

You can use a `for` loop to display every previous encryption activity.

```
for record in state.history:
    print("Original:", record.original_message)
    print("Encrypted:", record.encrypted_message)
    print("Key:", record.encryption_key)
    print("Date:", record.date_created)
```

In your actual Drafter history page, you would use the loop to create a list of displayed text components instead of using `print()`.

For example, conceptually:

```
history_items = []

for record in state.history:
    history_items.append(
        f"Encrypted Message: {record.encrypted_message}"
    )
```

This is useful because the user can encrypt multiple messages, and the website can display all of them without needing separate variables for each message.

8. Final route function list

To make the five pages and seven route functions fit together, I recommend this exact list:

#	Function	Type
1	`home_page(state)`	Page route
2	`encrypt_page(state)`	Page route
3	`decrypt_page(state)`	Page route
4	`history_page(state)`	Page route
5	`help_page(state)`	Page route
6	`encrypt_message(state)`	Encryption action
7	`decrypt_message(state)`	Decryption action

You can make `generate_random_key()` a regular helper function rather than another route.

9. How the overall program fits together

Here is the proposed structure of your Python program:

```
from dataclasses import dataclass, field

from drafter import *

import secrets

from datetime import datetime
```

```python
@dataclass
class EncryptionRecord:
    original_message: str
    encrypted_message: str
    encryption_key: str
    date_created: str


@dataclass
class State:
    current_message: str = ""
    current_key: str = ""
    current_result: str = ""
    history: list[EncryptionRecord] = field(default_factory=list)


# 1. Home page
def home_page(state: State) -> Page:
    ...


# 2. Encrypt page
def encrypt_page(state: State) -> Page:
```

```
    ...



# 3. Decrypt page
def decrypt_page(state: State) -> Page:
    ...



# 4. History page
def history_page(state: State) -> Page:
    ...



# 5. Help page
def help_page(state: State) -> Page:
    ...



# 6. Encrypt message
def encrypt_message(state: State) -> Page:
    ...



# 7. Decrypt message
```

```
def decrypt_message(state: State) -> Page:
    ...



# Helper function
def generate_random_key():
    return secrets.randbelow(25) + 1
```

This is a structural starting point, not a complete runnable application. You'll need to add your Drafter `@route` decorators, page components, and event handlers using the syntax supported by your course's Drafter version.

One technical consideration: The example key generator produces a number from 1 to 25, suitable for a simple Caesar cipher demonstration. A Caesar cipher is not secure for protecting real messages. If you want genuine security, use a vetted authenticated-encryption library and a cryptographically secure key generator.

10. Requirement checklist

Project requirements

Covered

Purpose and functionality described

Two dataclasses defined

Four State attributes, including a list of dataclass instances

Seven route/action functions identified

Five pages described

Four user input textboxes described

At least three if statements provided

For loop iterating over dataclass instances

Random key generation planned

My biggest recommendation is to prioritize the encryption and decryption functionality first. Once those work correctly, add the history page and visual improvements. That will give you a much better chance of finishing a polished application within three weeks.

why does EncryptionRecord not hold the decrypted message too

Good question! The reason I left `decrypted_message` out of `EncryptionRecord` is that it is usually unnecessary to store both the original message and its decrypted version.

Think about it:

- `original_message` is the message before encryption.
- `encrypted_message` is the message after encryption.
- `decrypted_message` is simply the original message recovered through decryption.

Therefore, if decryption works correctly, `decrypted_message` should be identical to `original_message`.

However, you can absolutely include it!

If you want your history to keep track of the decrypted result too, you can change your dataclass to:

```
@dataclass
```

```
class EncryptionRecord:

   original_message: str

   encrypted_message: str

   encryption_key: str

   decrypted_message: str

   date_created: str
```

My recommendation

I would actually keep `decrypted_message` out of `EncryptionRecord` and instead put it in your main `State` dataclass:

```
@dataclass

class State:

   current_message: str = ""

   current_key: str = ""

   current_encrypted_message: str = ""

   current_decrypted_message: str = ""

   history: list[EncryptionRecord] = field(default_factory=list)
```

Why?

Your `EncryptionRecord` represents a saved encryption activity, while `State` represents what the user is currently doing.

For example:

- The user encrypts a message → save it in `history`.

- The user decrypts a message → display the result in `current_decrypted_message`.
- The user starts another encryption → update the current fields without losing previous history.

This keeps your two dataclasses organized and avoids storing duplicate information.

shouldn't there be a help page route function

Yes! Your website should definitely have a `help_page()` route function if you're planning to include a Help page in your five-page design.

I included it in the seven-function list previously, but I should have made its role clearer.

Your seven route functions should be:

#	Route function	Purpose
1	`home_page(state)`	Main welcome page and navigation.
2	`encrypt_page(state)`	Allows users to enter messages and generate keys.
3	`decrypt_page(state)`	Allows users to decrypt messages using their keys.
4	`history_page(state)`	Displays previously encrypted messages.
5	`help_page(state)`	Explains how to use the website.
6	`encrypt_message(stat`	Processes and saves an encrypted

	e)	message.
7	decrypt_message(state)	Processes an encrypted message and displays the original.

What should the Help page contain?

I recommend including:

- What is encryption? A short explanation of converting readable messages into encrypted text.
- How to generate a key: Explain how the random key generator works.
- How to encrypt: Step-by-step instructions.
- How to decrypt: Explain why users need the original encryption key.
- Common errors: Explain what happens when users enter an empty message or incorrect key.

One important distinction

`help_page()` should be a page route, not an action function. It simply displays information and doesn't need to change the State.

That gives you five actual website pages and two processing functions, for seven functions total.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.